

Telemetry Format Converter

README | Technical Assessment Submission

Python 3 · JSON · Unit Tested

Overview

This project implements a telemetry data normalisation pipeline that converts device messages from two proprietary JSON formats into a single, unified output format. It was developed as part of the Deloitte technical assessment.

Project Structure

File	Description
<code>main.py</code>	Entry point. Contains conversion functions and unit tests.
<code>data-1.json</code>	Input sample — Format 1 (flat telemetry structure with pipe-delimited location).
<code>data-2.json</code>	Input sample — Format 2 (nested device object, ISO 8601 timestamp).
<code>data-result.json</code>	Target unified output format used to validate both conversions.

Input Formats

Format 1

A flat JSON structure where the device location is encoded as a slash-delimited string, and status/temperature fields use non-standard key names.

```
{ "deviceID": "...", "deviceType": "...", "timestamp": 1624445837783,
  "location": "japan/tokyo/keiyō-industrial-zone/...",
  "operationStatus": "healthy", "temp": 22 }
```

Format 2

A nested JSON structure with a separate device object, individual location fields, and an ISO 8601 timestamp string.

```
{ "device": { "id": "...", "type": "..." },
  "timestamp": "2021-06-23T10:57:17.783Z",
  "country": "japan", "city": "tokyo", ... }
```

Unified Output Format

Both input formats are converted into the following structure:

```
{ "deviceID": "...", "deviceType": "...", "timestamp": 1624445837783,
  "location": { "country": "...", "city": "...", "area": "...",
                "factory": "...", "section": "..." },
```

```
"data": { "status": "healthy", "temperature": 22 } }
```

Implementation Details

convertFromFormat1

- Splits the location string on '/' into 5 named fields.
- Remaps operationStatus → data.status and temp → data.temperature.
- Timestamp is already in milliseconds since epoch — passed through unchanged.

convertFromFormat2

- Parses the ISO 8601 timestamp using datetime.strptime and converts to milliseconds since Unix epoch.
- Extracts deviceId and deviceType from the nested device object.
- Constructs the location object from the flat country/city/area/factory/section fields.
- Copies the data object directly (keys already match the unified format).

Format Detection

The main() function detects the input format by checking for the presence of the device key. If absent, Format 1 is assumed; otherwise Format 2 is used.

How to Run

Prerequisites

- Python 3.x installed
- No external dependencies — uses only Python standard library (json, unittest, datetime)

Running the Tests

```
python main.py
```

Expected output:

```
test_dataType1 ... ok
test_dataType2 ... ok
test_sanity     ... ok
-----
Ran 3 tests in 0.004s
OK
```

Test Cases

Three unit tests are included in the TestSolution class:

Test	Purpose
test_sanity	Verifies the expected result JSON is valid and self-consistent.
test_dataType1	Converts data-1.json and asserts output matches data-result.json.

`test_dataType2`

Converts data-2.json and asserts output matches data-result.json.

Key Design Decisions

- Timestamp conversion uses UTC-aware datetime arithmetic to avoid timezone errors.
 - `round()` is used instead of `int()` when converting timestamp to milliseconds to prevent off-by-one errors from floating-point precision.
 - Format detection is based on the presence of the device key, which is unique to Format 2.
 - No third-party libraries are used, keeping the solution portable and dependency-free.
-